Built Like a System.

Gym Membership Management System

Team: "GymTech"

E2 – Software Modeling and Design Specifications

Academic Year 2025-2026

1. UML class Diagram -> "What is the system made of?"

The **UML Class Diagram** below presents the main classes of the **Gym Membership Management System**, including their **attributes, methods,** and **relationships** (associations, aggregations).

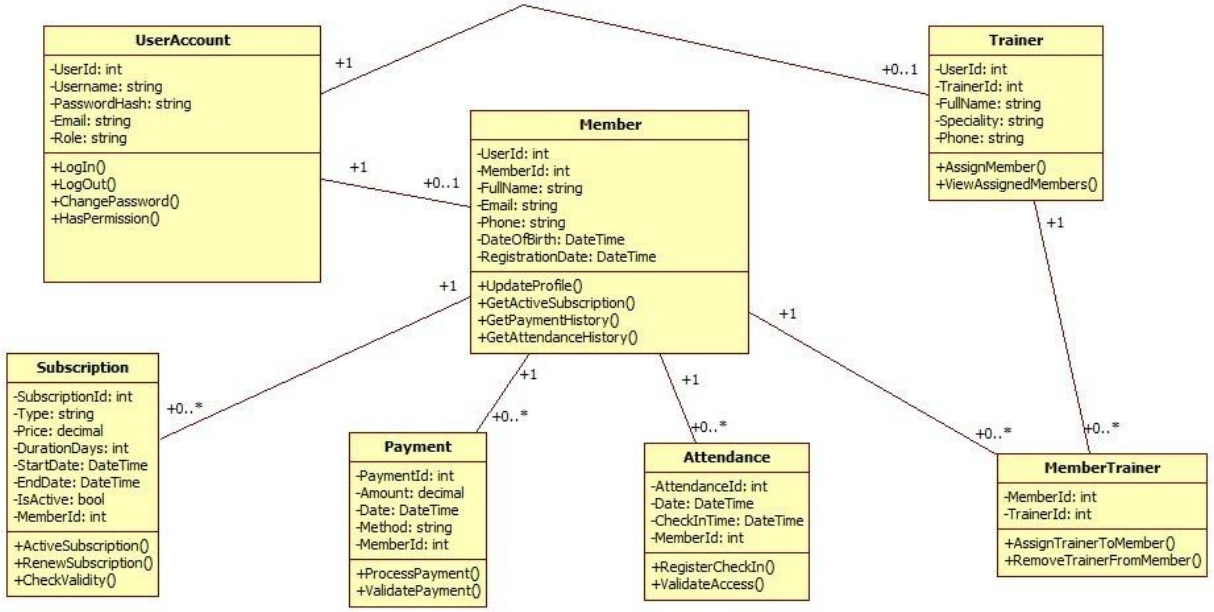


Figure 1 – UML Class Diagram

We added a **UserAccount class** to handle login and roles, like **Admin, Receptionist, Trainer, and Member.** Depending on the role, a user can be linked to a Member or a Trainer, but not necessarily to both. The main functionality is around Members, Trainers, Subscriptions, Payments, and Attendance. A member can have multiple subscriptions, payments, and attendance records, which is why we use **one-to-many relationships.** The connection between Members and Trainers is **many-to-many**, so we used an extra class called MemberTrainer to manage that. We didn't use inheritance because each class has a different role in the system and they don't really extend each other.

Key relationships:

- **UserAccount – Member / Trainer:** One-to-one: each member or trainer has exactly one user account for authentication.
- **Member – Subscription:** One-to-many: a member can have multiple subscriptions over time.
- **Member – Payment:** One-to-many: a member can make multiple payments.
- **Member – Attendance:** One-to-many: a member can have multiple attendance records.
- **Member – MemberTrainer:** Many-to-many through MemberTrainer: a member can be assigned to multiple trainers.
- **Trainer – MemberTrainer:** Many-to-many through MemberTrainer: a trainer can be assigned to multiple members.

2.0 Application Architecture Description -> "How is the system built?"

The system follows **a 3-tier layered architecture** separating concerns into **Presentation, Business Logic, and Data Access layers**, ensuring modularity, maintainability and separation of responsabilities.

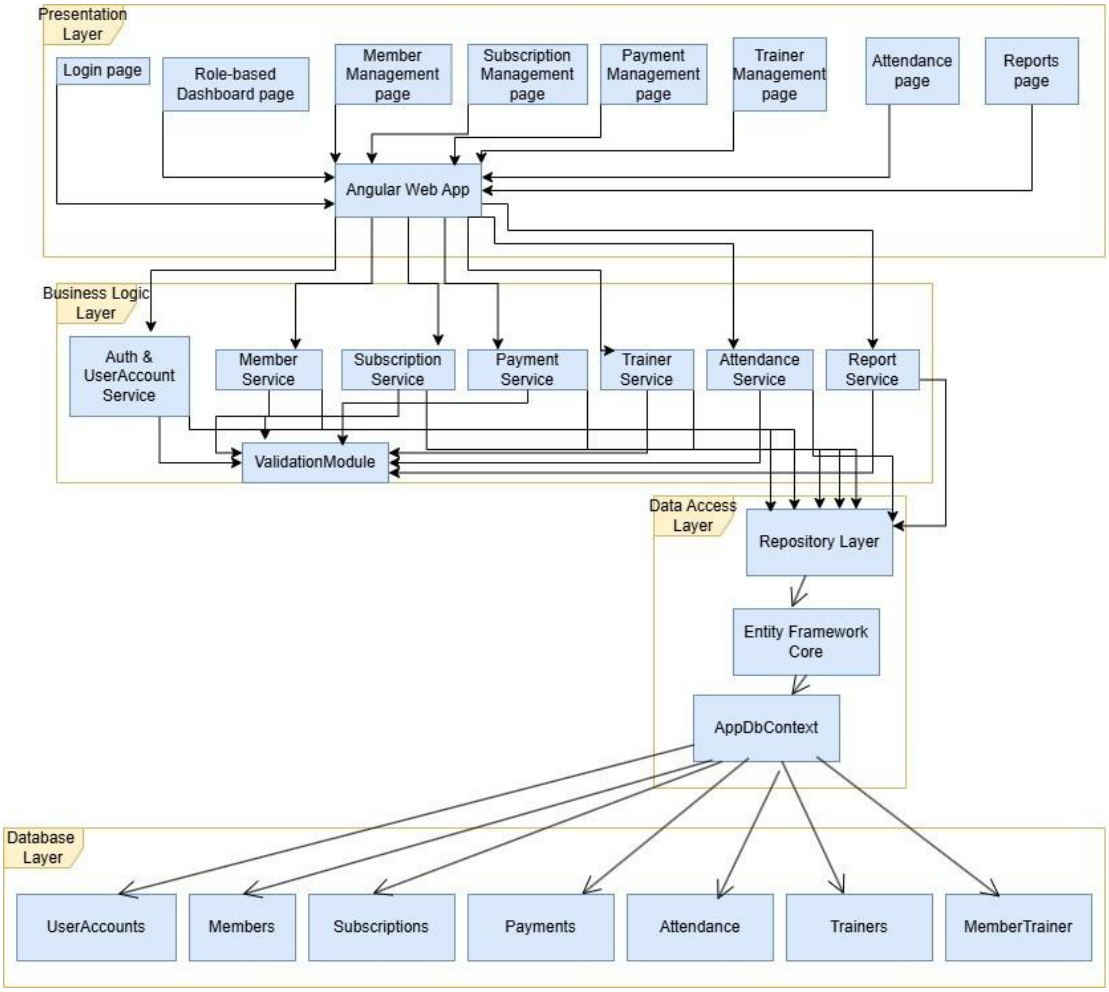


Figure 2 – Architecture Diagram

Layer descriptions:

- **Presentation Layer (Angular Web App):** Contains all UI pages: Login, Role-based Dashboard, Member Management, Subscription Management, Payment Management, Trainer Management, Attendance, and Reports.
- **Business Logic Layer (.NET Services):** Contains the core services: Auth & UserAccount Service, Member Service, Subscription Service, Payment Service, Trainer Service, Attendance Service, and Report Service. All services interact with a shared ValidationModule to ensure data integrity.
- **Data Access Layer (Repository + EF Core):** Implemented using the Repository Pattern on top of Entity Framework Core. The AppDbContext manages the connection and entity mapping to the SQL Server database.

- **Database Layer (SQL):** SQL Server database with tables: UserAccounts, Members, Subscriptions, Payments, Attendance, Trainers, MemberTrainer.

2.1 Entity-Relationship Diagram (ERD) -> "How is the data stored?"

The ERD below shows the database structure with all **entities, primary keys, foreign keys,** and **relationships** between tables.

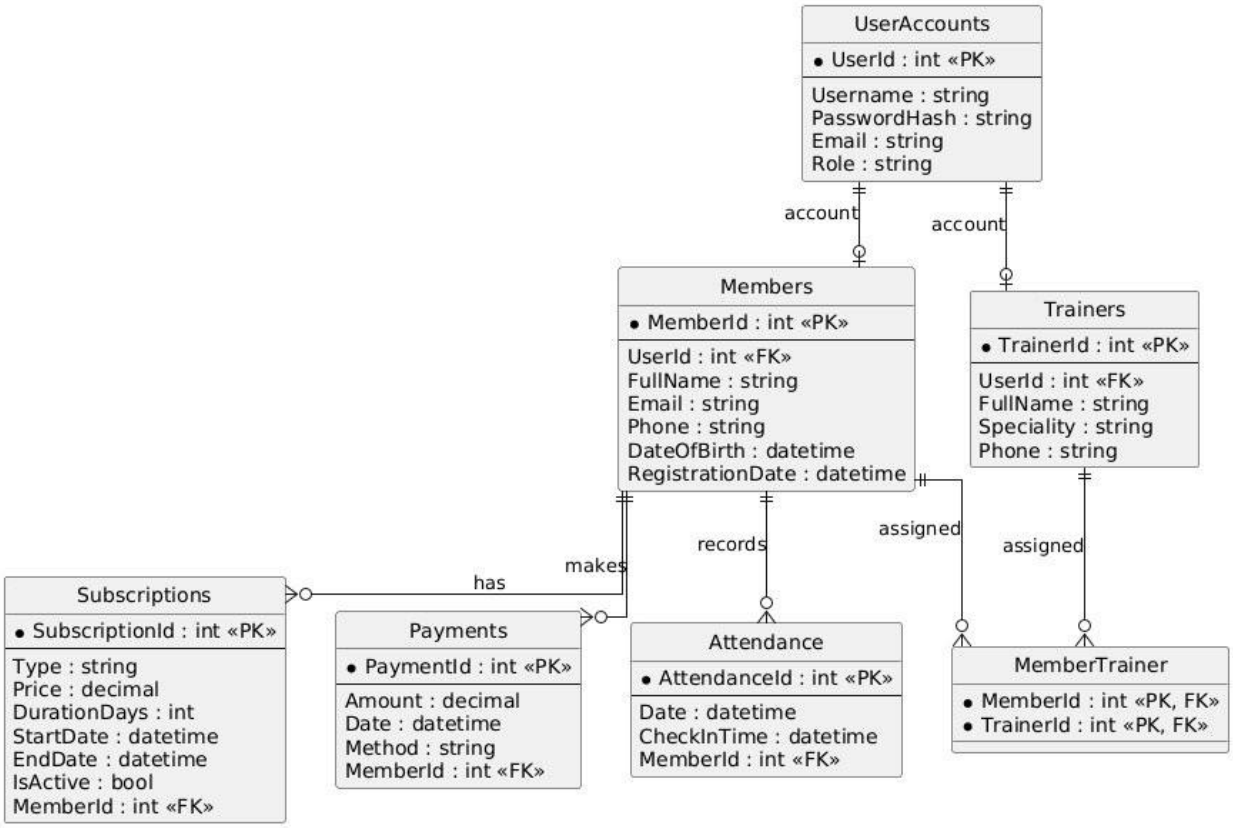


Figure 3 – Entity-Relationship Diagram (ERD)

2.2 Use Case Diagram -> "What does the system do?"

The **Use Case Diagram** illustrates the **interactions between the system actors** (Admin, Receptionist, Trainer, Member) **and** the available system **functionalities.**

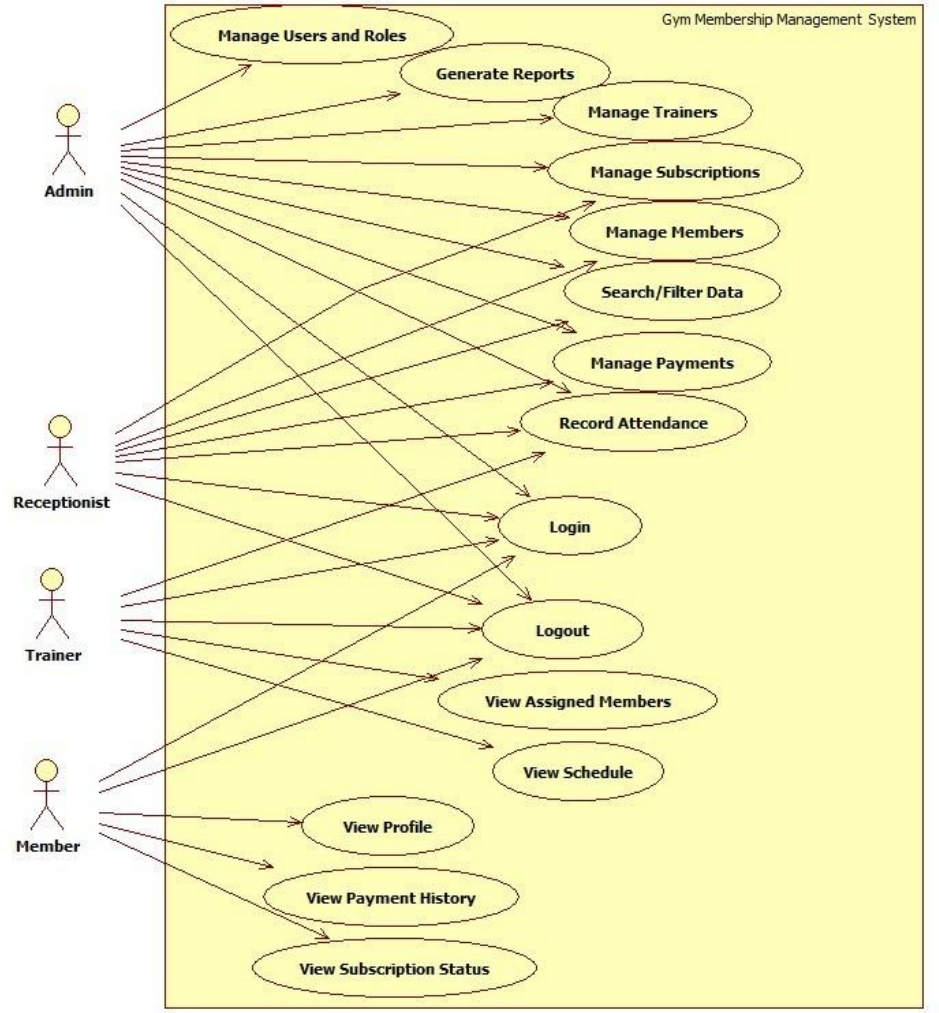


Figure 4 – Use Case Diagram

Actor descriptions:

- **Admin:** Manage members, trainers, subscriptions, payments, attendance, reports, and users/roles.
- **Receptionist:** Add and update members, view subscriptions, register payments, and check attendance.
- **Trainer:** View assigned members, view schedule, and record attendance/sessions.
- **Member:** View personal profile, subscription status, and payment history.

3. Graphical User Interface Design

The **user interface** is designed using **Angular with Figma mockups** as the design reference. The application includes the following main windows/pages:

The GymTech application features a modern, responsive user interface with a dark theme and pink accents. The UI is optimized for different user roles (Admin, Member, Trainer, Receptionist) with role-specific dashboards and navigation menus. All interfaces follow consistent design principles ensuring professional appearance and intuitive user experience.

3.1 Sketches and Mockups of Application Windows

A. Authentication Pages

Login Page

Route: `/login`

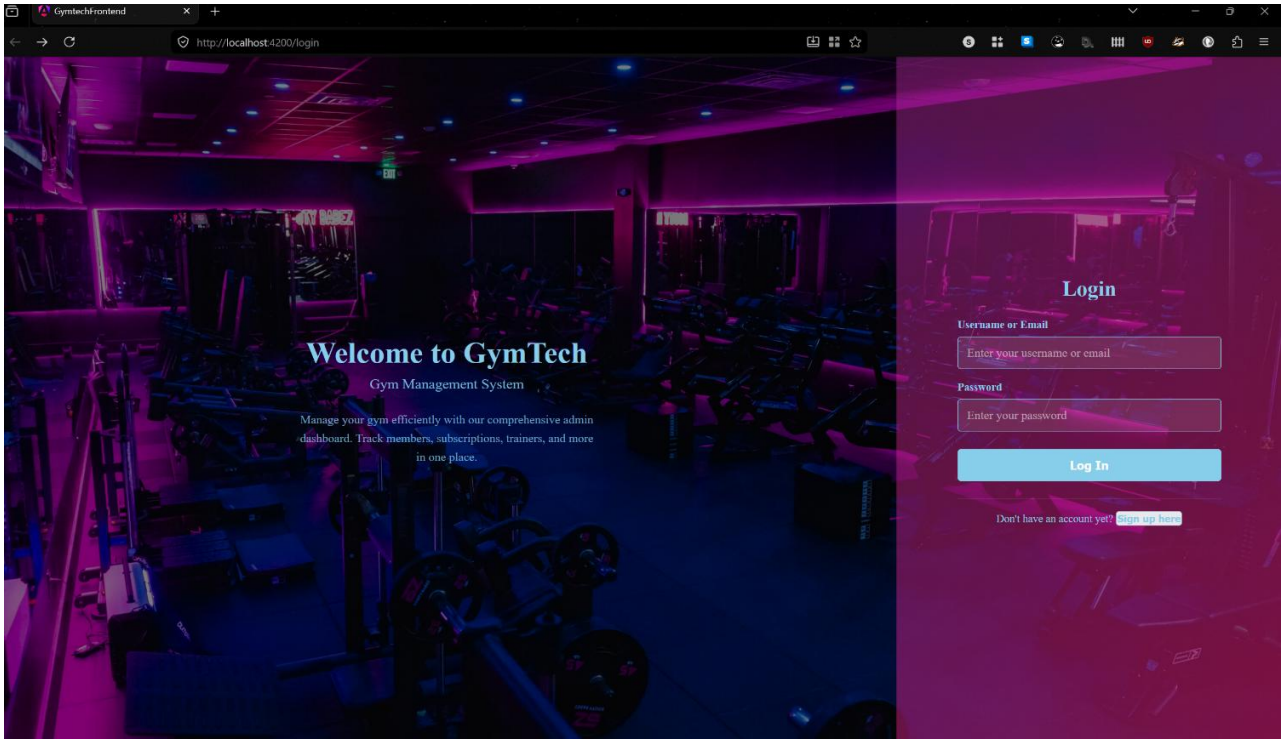
Layout: Two-column layout with gym background image on left, login form on right

Components:

- Welcome panel with application branding and description (70% width)
- Login form with username/email and password fields (30% width)
- "Log In" button with pink accent (#e91e63)
- "Sign up here" link for new user registration

Color Scheme: Dark theme (#1a1a1a) with pink accents (#e91e63)

Description: Provides secure entry point for authenticated users with clear visual hierarchy and intuitive form layout.



Sign-up Page

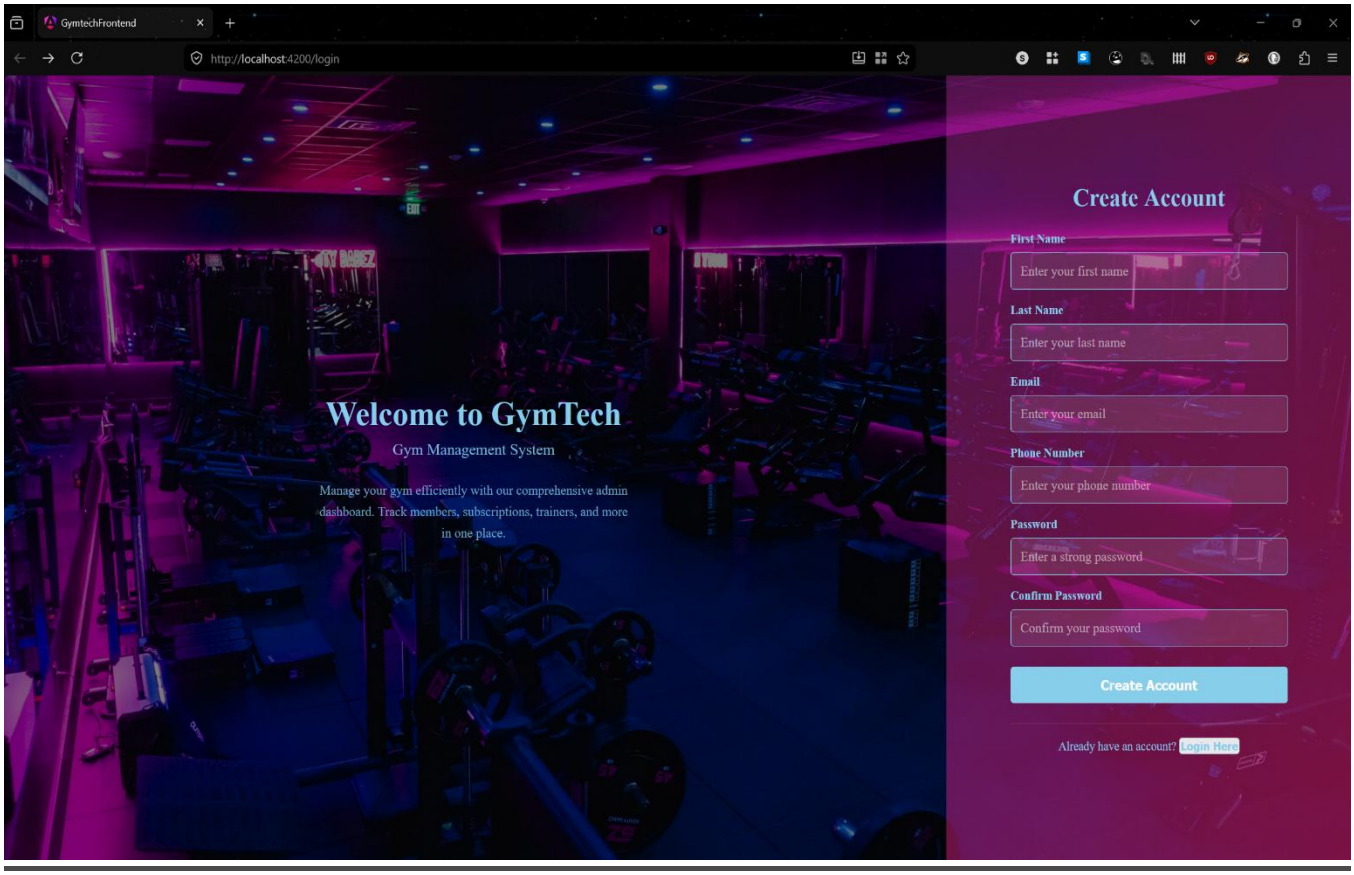
Route: `/login` (signup mode)

Layout: Same two-column layout as login page

Components:

- Registration form with 6 input fields: First Name, Last Name, Email, Phone, Password, Confirm Password
- "Create Account" button
- "Login Here" link for existing users
- Client-side form validation

Description: Allows new users to register with comprehensive account information collection and validation.



B. Dashboard Pages (Role-Based)

Admin Dashboard

Route: `/dashboard`

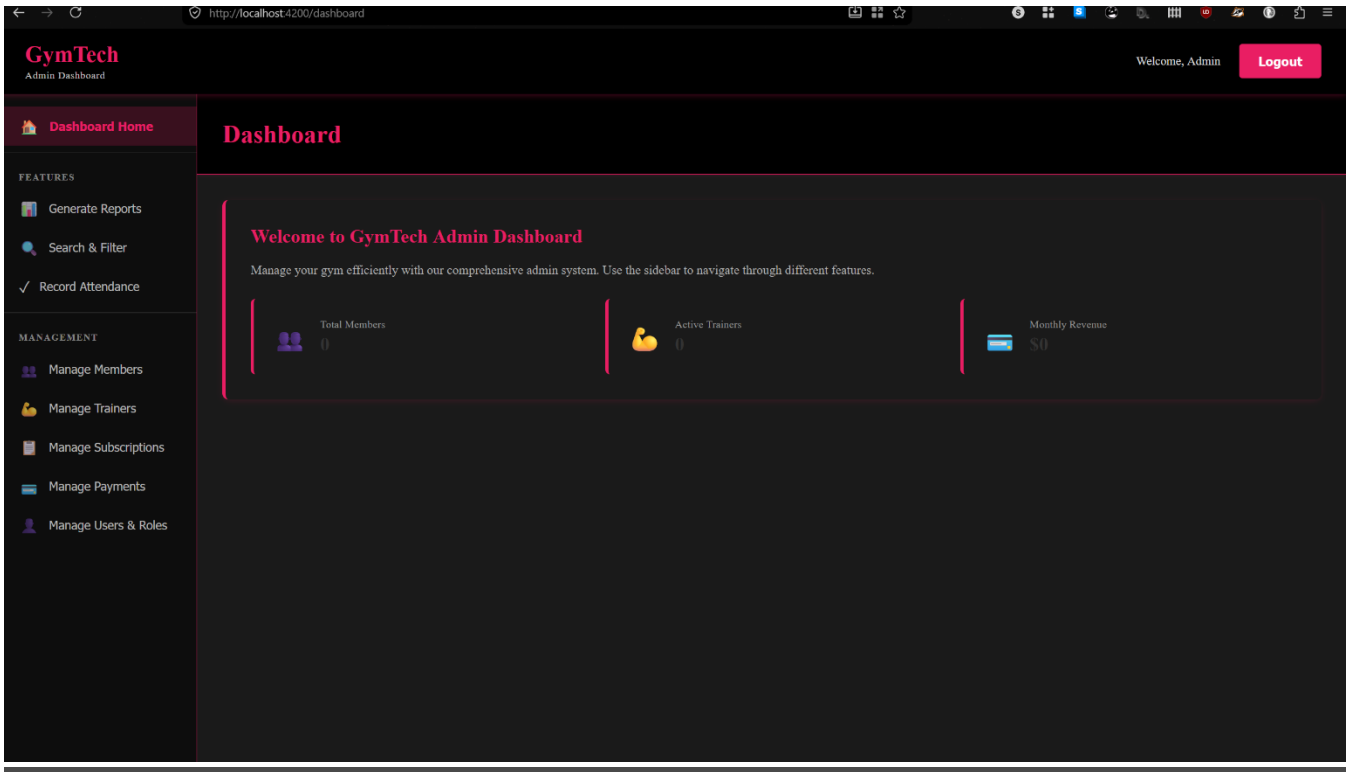
Target User: Administrator

Layout Components:

- Top Navigation Bar: Logo, dashboard type, user greeting, logout button
- Left Sidebar: Feature navigation with icon-based menu items
- Main Content Area: Welcome card with system overview
- Stat Cards: Total Members, Active Trainers, Monthly Revenue

Navigation Sections: Dashboard Home, Generate Reports, Search & Filter, Record Attendance, Manage Members, Manage Trainers, Manage Subscriptions, Manage Payments, Manage Users & Roles

Description: Central hub for gym administrators to manage all system features and monitor key metrics. Complete access to all features and data.



Member Dashboard

Route: `/member-dashboard`

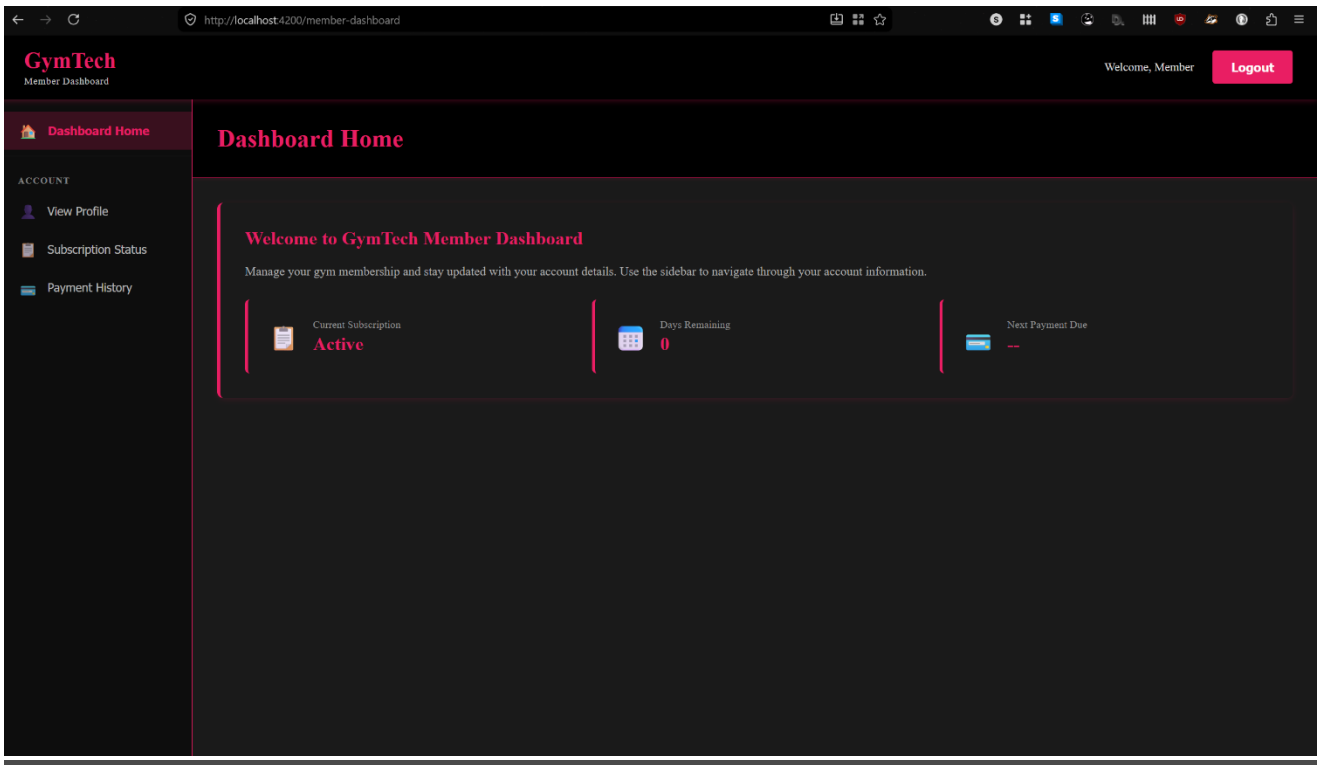
Target User: Gym Member

Layout Components:

- Simplified navigation sidebar
- Welcome card with membership information
- Stat Cards: Current Subscription, Days Remaining, Next Payment Due

Navigation Sections: Dashboard Home, View Profile, Subscription Status, Payment History

Description: Personal dashboard for members to track membership status, view subscription details, and manage account information.



Trainer Dashboard

Route: `/trainer-dashboard`

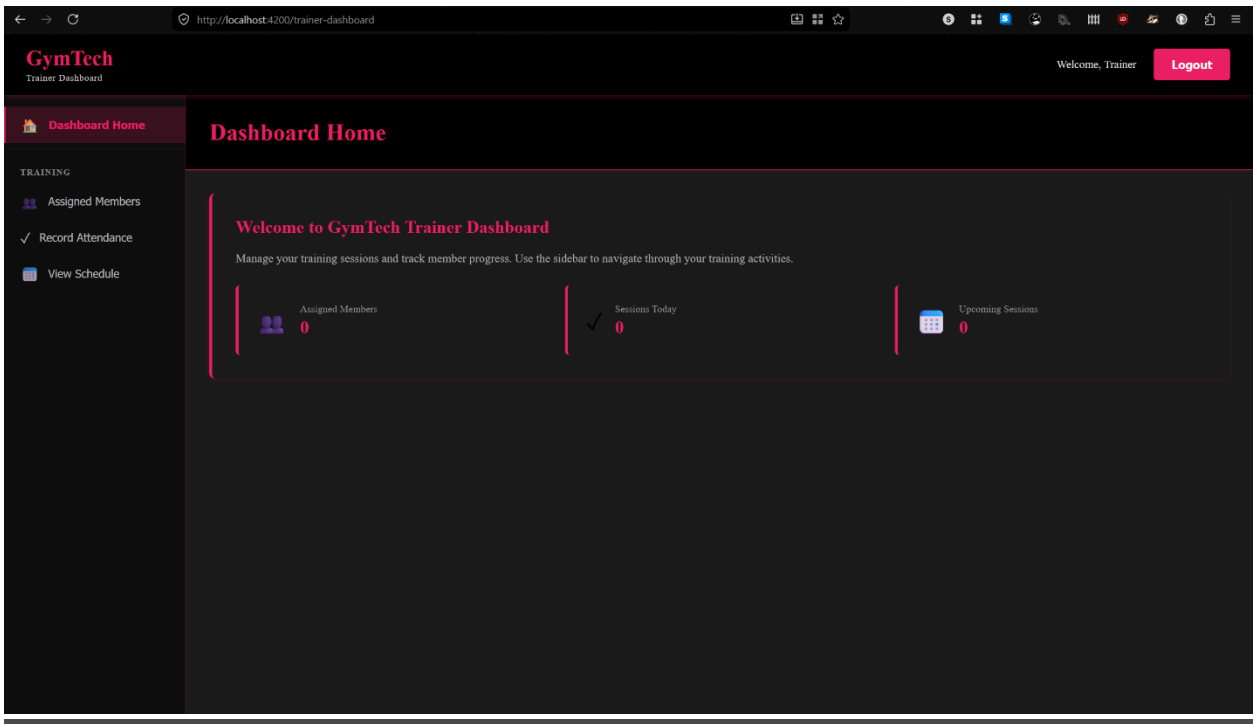
Target User: Trainer

Layout Components:

- Training-focused navigation sidebar
- Training metrics display
- Stat Cards: Assigned Members, Sessions Today, Upcoming Sessions

Navigation Sections: Dashboard Home, Assigned Members, Record Attendance, View Schedule

Description: Specialized interface for trainers to manage assigned members, schedule sessions, and track training activities.



Receptionist Dashboard

Route: `/receptionist-dashboard`

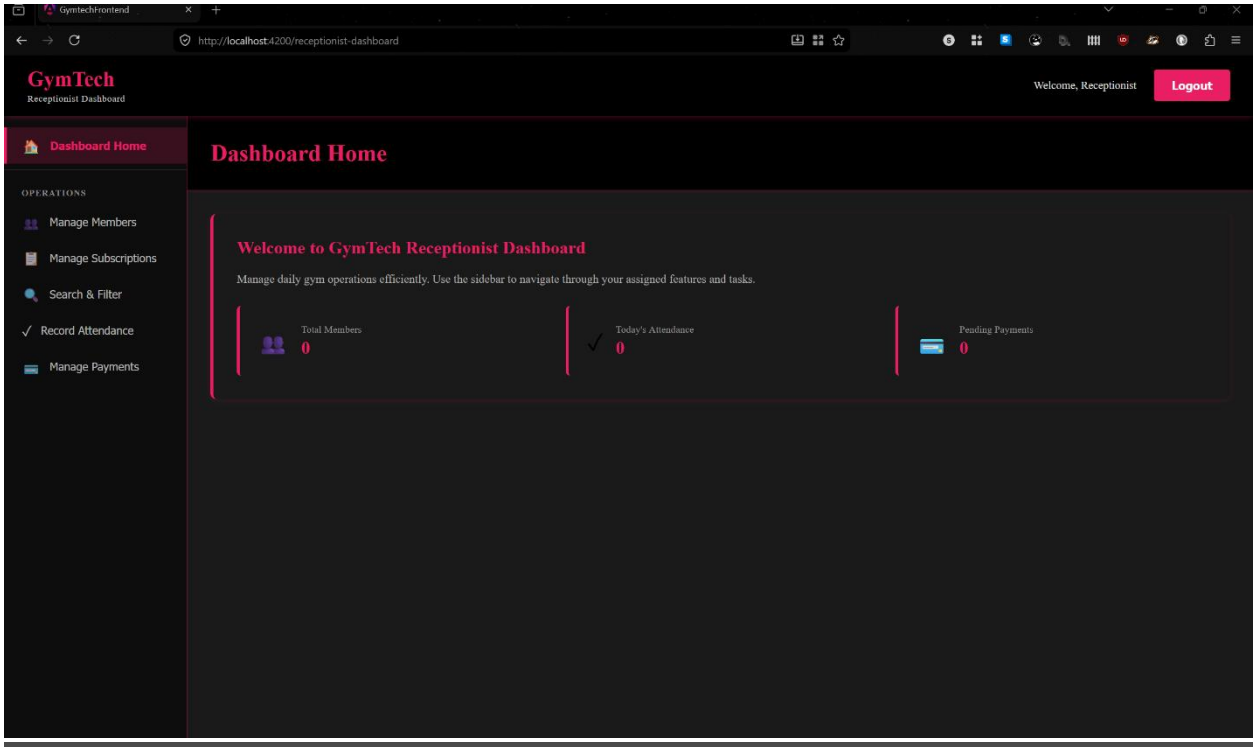
Target User: Receptionist

Layout Components:

- Operations-focused navigation sidebar
- Daily operational metrics
- Stat Cards: Total Members, Today's Attendance, Pending Payments

Navigation Sections: Dashboard Home, Manage Members, Manage Subscriptions, Search & Filter, Record Attendance, Manage Payments

Description: Operational interface for receptionists to manage daily gym activities, member check-ins, and payment processing.



3.2 Design System Specifications

Color Palette

Color Name	Hex Code	Usage
Primary Pink	#e91e63	Accents, highlights, active states, CTAs
Navigation Black	#000000	Top navigation bar background
Sidebar Dark	#0d0d0d	Sidebar background
Content Dark	#1a1a1a	Main content area background
Text White	#ffffff	Primary text color
Text Gray	#999999	Secondary/muted text

Typography

- **Page Titles:** 2rem, Bold, #e91e63
- **Section Titles:** 1.5rem, Bold, #e91e63
- **Body Text:** 0.95-1rem, Regular, #b3b3b3
- **Labels:** 0.75-0.85rem, Bold, #999999
- **Stat Values:** 1.5rem, Bold, #333333

---

4. Description of the Main Classes

The following table describes **each main class**, its role in the system, and its key responsibilities:

Class	Role	Responsibilities
UserAccount	Authentication & Access Control	Manages user credentials and roles (Admin, Receptionist, Trainer, Member). Handles login, logout, password management, and access permissions.
Member	Core Domain Entity	Represents a gym member. Stores personal details and links to subscriptions, payments, attendance records, and assigned trainers.
Subscription	Membership Management	Tracks the type, price, duration, and validity of a member's subscription. Ensures only active subscriptions are used.
Payment	Financial Transactions	Records payment transactions made by members. Processes and validates payments linked to subscriptions.
Attendance	Check-in Tracking	Records each gym visit by a member. Validates access based on active subscription status.
Trainer	Trainer Management	Stores trainer details and specialization. Allows viewing and managing assigned members.
MemberTrainer	Many-to-Many Relationship	Junction entity linking members to their assigned trainers. Manages assignment and removal of trainers for members.

5. Test Plan

The test plan defines the **key test cases for the system**. Each test case specifies the feature being tested, the **input values** used, and the **expected result**. Tests are defined by the QA Engineer (Luidort Zsuzsa).

TC#	Feature	Input Values	Expected Result
FE-01	Login	Verify the component initializes correctly without errors.	expect(component).toBeTruthy() is true.
FE-02	Login	Check that the login form is the first thing a user sees.	currentPage is 'login' and title contains "Login".
FE-03	Login	Ensure standard username and password fields exist in the HTML.	input[type="text"] and input[type="password"] are present.
FE-04	Login	Simulate the login action and check if it targets the dashboard.	routerStub.lastNavigatedUrl matches '/dashboard'.
FE-05	Signup	Call goToSignup() and check if the form title changes.	Form title updates to "Create Account".
FE-06	Signup	Verify all 6 registration fields are rendered in the DOM.	All IDs (#signup-firstname, #signup-email, etc.) are found.
FE-07	Signup	Verify that every signup field is marked as required.	hasAttribute('required') is true for all 6 fields.
FE-08	Signup	Click "Login Here" on the signup form to go back.	currentPage resets to 'login' and title reflects the change.
FE-09	Dashboard	Test navigation between Admin specific features (Reports, Members, etc.).	activeFeature updates and getPageTitle() returns matching title.
FE-10	Dashboard	Check visibility of member-specific cards (Subscription, Days Remaining).	Specific stat-cards are present in the DOM.
FE-11	Dashboard	Verify unique headers and stat cards for these specific roles.	Displays "Assigned Members" (Trainer) or "Attendance" (Receptionist).
FE-12	Global Dashboard	Trigger the logout function from any dashboard.	User is redirected to '/login' and console logs logout.
FU-01	UI	Manual: Try to login or sign up with all fields empty.	Browser blocks the action with a "Please fill out this field" tooltip.
FU-02	UX	Manual: Enter an invalid email address format (e.g., "adminatgym").	HTML5 validation prevents submission and requests an '@' symbol.
FU-03	UX	Manual: Click "Coming Soon" features like Reports or Payments.	UI displays the appropriate "Coming Soon" empty-state message.
FU-04	UI	Manual: Change browser window	Sidebar and cards re-align or stack to fit the smaller screen.
FU-05	Security	Manual: Type into the password field.	Characters are obscured (dots/stars) for privacy.

BE-01	MemberService	Retrieves an existing member by ID	Returns Member object
BE-02	MemberService	Queries for an ID that doesn't exist	Returns null
BE-03	MemberService	Adds a new member to the context	Returns Member object
BE-04	MemberService	Updates fields for an existing member	State marked as Modified
BE-07	MemberService	Deletes a member record	Record removed
BE-06	MemberService	Attempts to delete a non-existing ID	No error (No-op)
BE-08	MemberController	Retrieves member for valid ID	Returns 200 OK
BE-09	MemberController	Queries for invalid ID	Returns 404 Not Found
BE-10	MemberController	Adds member via POST	Returns 201 Created
BE-11	MemberController	Deletes member via DELETE	Returns 204 No Content
BE-12	Member	Input invalid email format (e.g., "name@com")	Validation fails; returns error
BE-13	Member	Input non-numeric phone or length $\neq$ 10	Validation fails; returns error
BE-14	Member	Input a future date for birthday	Validation fails; returns error
BE-15	AdminUser	Verify format and required status for Admin	Validation fails if empty or malformed
BE-16	AdminUser	Input numeric value instead of string for Name	Validation fails; type mismatch error
BE-17	Trainer	Input empty string for trainer name	Validation fails; "Required" error
BE-18	Trainer	Input phone number with letters	Validation fails; format error
BE-19	Subscription	Set EndDate earlier than StartDate	Validation fails; logical date error
BE-20	Subscription	Input negative number for DurationDays	Validation fails; range error
BE-21	Payment	Input negative value (e.g., -50.00)	Validation fails; must be positive
BE-22	Payment	Input invalid date format or future date	Validation fails; format/range error
BE-23	Common	Verify name fields are valid strings	Validation fails on non-string input

*FE – frontend tests

*FU – frontend user tests

*BE – backend tests

6. Other Relevant Information

The application is planned as a **role-based Gym Membership Management System**. Different user roles will have different access rights inside the application:

- Admin – manages the entire system;
- Receptionist – manages members and subscriptions;
- Trainer – views assigned members and schedule;
- Member – views personal information and subscription details.

The project uses a **layered structure** with:

- Angular for the frontend,
- ASP.NET Core Web API for the backend,
- Entity Framework Core for database communication,
- SQL Server for data storage.

GitHub is used for collaboration and task management between team members, while **Swagger** is used for testing the backend endpoints during development.

At the current stage, the main diagrams, database structure, and core backend entities are completed. Frontend pages and dashboard mockups are also prepared for the next implementation steps.